

SAKSHYA · साक्ष्य

Evidence & Action Navigator for Online Harassment & Image-Based Abuse

She Solves 3.0 Hackathon | Complete Architectural Guide & Judge Defense Manual

Executive Summary: Sakshya is a privacy-first, client-side progressive web application designed for women and girls in India facing non-consensual intimate image dissemination, deepfakes, cyberstalking, and digital extortion. It takes a victim from acute trauma and panic to a legally grounded, court-ready Action Pack: on-device cryptographic evidence hashing, tamper-evident timeline generation, local multilingual OCR, statutory retrieval with verified India Code citations, and Section 63 BSA certificate drafting. Files never leave the user's browser, the backend is 100% stateless, and guidance is strictly cited and non-hallucinatory.

1. The 5 Non-Negotiable Privacy Tenets

Victims of image-based abuse cannot risk their private photos or screenshots leaking to a cloud database, an LLM training pool, or a server log. Sakshya enforces 5 absolute architectural boundaries:

Tenet	Architectural Enforcement & Proof
1. Zero Image Uploads	Original files and screenshots never leave the browser. Web Crypto API computes SHA-256 digests in memory. Only the 64-char hex hash and metadata are recorded. The backend rejects any non-JSON or multipart payload with HTTP 415.
2. Stateless Backend	Zero database (no SQLite/PostgreSQL/MongoDB). Request bodies are never logged or stored. Python logging is suppressed with logging.disable(CRITICAL). Headers enforce Cache-Control: no-store, nosniff, and no-referrer.
3. Zero LLM Image Exposure	Images never reach an AI model. Optical Character Recognition (OCR) executes entirely on-device using Tesseract.js WebAssembly. Only user-reviewed, edited text is sent to extraction endpoints.
4. 100% Sourced Citations	Every legal provision returned is strictly retrieved from an audited corpus of 18 sections (BNS, IT Act, BSA Section 63, POCSO). Every claim flags needs_verification: true and links directly to official India Code acts. If no match exists, the system explicitly states so.
5. Ubiquitous Disclaimer	The prominent notice 'Guidance, not legal advice' appears persistently across every screen, dialog, PDF export header, and canvas footer. Sakshya assists preparation; it does not replace an advocate.

2. Site Flow & Page-by-Page Breakdown

Sakshya is architected as an intuitive 6-stage progressive workflow designed with trauma-informed UX principles, ensuring users never feel overwhelmed.

Screen 1: Landing Page (Safe Entry)

Features: Calming hero section, 'You deserve to feel safe' reassurance, immediate 'Get help now' action, overview of the 3-step path, and prominent security card explaining that data stays on device.

Security Controls: Persistent Quick Exit button in header; language switcher (English, Hindi, Marathi); skip link for accessibility; zero external tracking or analytics scripts.

Screen 2: Triage Flow (Safety & Urgency Filter)

Features: 3-question progressive wizard:

1. Incident category: Leaked/deepfaked images, threats/blackmail, stalking, or impersonation.
2. Immediate danger check: If 'Yes', instantly shifts to emergency mode displaying direct dial links to 112 (Emergency), 181 (Women Helpline), and 1098 (Childline).
3. Minor status check: If affected person is under 18, routes to specialized POCSO guidance, highlights Take It Down (NCMEC), and cites POCSO Act Sections 11 & 19.

Screen 3: Evidence Vault (Tamper-Evident SHA-256 Chain)

Features: Client-side evidence inventory. Users drag-and-drop screenshots/files. The browser reads bytes, hashes via Web Crypto SHA-256, and constructs an immutable ledger.

Hash Chain Architecture: Each record contains index, timestamp, fileName, fileHash, platform, note, prevHash, entryHash.

Genesis previous hash is 64 zeros. Entry hash is SHA256(index|timestamp|fileName|fileHash|platform|note|prevHash).

Encryption: IndexedDB is encrypted with AES-GCM-256 derived via PBKDF2 (200,000 iterations, SHA-256) from user's secret passphrase.

Demo & Verification: 'Verify Chain' recalculates all hashes; 'Simulate Tampering' deliberately modifies an encrypted block to visually show how broken links are pinpointed (e.g., 'Broken at entry 1'). Downloadable Timeline PDF.

Screen 4: Incident Details & Local In-Browser OCR

Features: Users upload chat screenshots for local text extraction. Uses bundled Tesseract.js WebAssembly with pre-cached English, Hindi, and Marathi trained data. Runs 100% offline without sending pixels anywhere.

Extraction Endpoint: Cleaned text is submitted to POST /api/extract, which parses usernames (@handle), dates, threat categories, and a concise summary. The form populates automatically, allowing the user to review, edit, or redact every single field.

Screen 5: Legal Guidance (Grounded RAG Pipeline)

Features: Submits case summary to POST /api/guidance. Uses Sentence-Transformers cosine similarity backed by a strict keyword eligibility gate. Returns applicable provisions from the 18-section statutory corpus.

Card Details: Each provision displays the Act, Section number, Title, Plain-language summary, 'Why it applies' rationale, 'Needs verification' warning badge, and a direct hyperlink to India Code.

Reporting Routes: Direct links to National Cyber Crime Reporting Portal (cybercrime.gov.in), StopNCII.org (for adults), Take It Down (for minors), and major platform grievance reporting URLs (Instagram, WhatsApp, Facebook, X, Telegram, Snapchat, YouTube).

Screen 6: Action Pack & Document Generator

Features: Generates 3 court-ready, customized documents via POST /api/draft and POST /api/pdf:

1. Police / Cybercrime Complaint: Addressed to SHO/Cyber Cell, structured incident facts, safe contact, and evidence inventory.
2. Platform Takedown Request: Grievance officer request citing non-consensual sharing and IT Rules preservation notice.
3. Section 63 BSA Electronic Evidence Certificate: Complete preparation worksheet formatted per Bharatiya Sakshya Adhiniyam, 2023 Schedule (Part A: Party in lawful control; Part B: Expert hash verification).

Live Editing: Users can edit any text in the preview box before generating the final ReportLab PDF.

Screen 7: Next Steps & Local Action Checklist

Features: Trauma-informed recovery milestones divided into actionable time horizons: Next 1 Hour, Next 24 Hours, and This Week. Checkboxes persist locally in localStorage without storing sensitive case facts.

Global Features: Quick Exit & Helplines Modal

Quick Exit: Pressing Escape twice within 1.2 seconds or clicking 'Quick exit' instantly locks the session, clears memory, and executes `window.location.replace('https://www.google.com/')`.

Support Dialog: Accessible from any screen via 'Get support' button, listing emergency helplines and reporting resources.

3. Technology Stacks & Architectural Justifications

Every library and framework in Sakshya was chosen to satisfy strict privacy constraints, zero-cloud dependency, fast hackathon demonstration, and resilience against surveillance.

Frontend Technology Stack

Technology	Role	Why Chosen & Justification
Next.js 15 (App Router)	Application Framework	Enables fast production asset bundling, zero-latency client-side navigation, and seamless static exports suitable for local or edge hosting.
TypeScript 5.8	Type Safety & Schemas	Guarantees strict end-to-end interface parity with backend Pydantic models. Eliminates runtime null pointer errors during sensitive evidence handling.
Tailwind CSS 4	Design System & UI	Permits creating a calming, trauma-informed palette (forest green #173f35 and warm paper #f6f7f1), high contrast accessible focus states, and responsive styling down to 375px mobile screens.
Dexie.js (IndexedDB)	Encrypted Client Storage	Provides reliable, structured client-side storage for encrypted timeline records without requiring any remote database or third-party cookies.
Web Crypto API	Hardware-Accelerated Cryptography	Standard browser API executing PBKDF2 (200,000 iterations, SHA-256) and AES-GCM-256 encryption. Fast, native, and guarantees cryptographic operations stay in browser memory.
Tesseract.js (WASM)	In-Browser Client OCR	Runs optical character recognition directly on device WebAssembly threads with pre-cached English, Hindi, and Marathi models. Ensures user screenshots never touch any network API.
Custom i18n Context	Multilingual Localization	Instant, state-based translation across English, Hindi, and Marathi with zero URL routing, avoiding page reloads or leaking user language state to web servers.
PWA / Service Worker	Offline Resilience	Pre-caches 39 static application assets, icons, and WASM/OCR cores. Allows victims to reload the app, unlock their vault, and verify evidence completely offline.
Vitest & Playwright	Automated Testing	Vitest tests pure cryptographic hash chain logic; Playwright automates end-to-end browser user journeys, locale switches, tamper detection, and mobile responsiveness.

Backend Technology Stack

Technology	Role	Why Chosen & Justification
Python 3.11+ / FastAPI	Stateless API Framework	Asynchronous ASGI server offering sub-millisecond response times, automated Pydantic schema validation, and minimal memory footprint.
Pydantic v2 (StrictModel)	Input Sanitization	Configured with extra='forbid' and max_length bounds. Immediately rejects unexpected fields (e.g. uploaded images or tokens) and scrubs error messages to avoid reflecting user input.
Sentence-Transformers & TF-IDF	Dual-Mode Legal RAG	Embeds queries with paraphrase-multilingual-MiniLM-L12-v2 using NumPy cosine similarity. If the model is absent or offline, seamlessly falls back to Scikit-Learn character TF-IDF with zero external calls.
ReportLab & UHarfBuzz	Court-Ready PDF Engine	Builds deterministic in-memory PDF documents (Complaints, Takedowns, BSA 63). Uses UHarfBuzz for proper Indic/Devanagari text shaping and Windows Nirmala UI font fallback.
PrivacyBoundary Middleware	Zero-Leakage Network Shield	Custom ASGI middleware that enforces a 2 MB request ceiling, limits content-type strictly to application/json, and injects Cache-Control: no-store into all responses.
Pluggable LLM Adapter	Deterministic & Cloud AI	Defaults to a zero-cloud, rule-based Mock provider that works with no API key. Can be toggled to OpenAI, Anthropic, or Gemini for text summarization while strictly forbidding the LLM from inventing statutes.

4. How to Pitch Sakshya to Judges (The 3-Minute Script)

Judges evaluate three things: Empathy for the Problem, Technical Elegance, and Real-World Defensibility. Here is the ideal pitch narrative:

Step 1: The Problem (0:00 - 0:35)

'Good morning judges. When an Indian woman discovers her private photos leaked or deepfaked on Telegram or Instagram, her first reaction is acute panic. She deletes the chat, blocks the abuser, and closes her phone. In doing so, she destroys the very evidence the police need under the new Bharatiya Sakshya Adhiniyam, 2023. If she seeks help online, current tools ask her to upload her private images to cloud servers or type into ChatGPT, leaking her data even further. We built Sakshya (साक्ष्य) — an Evidence and Action Navigator designed to be her private, calm, digital first responder.'

Step 2: The Core Tech & Privacy Innovation (0:35 - 1:20)

'Sakshya operates on a non-negotiable principle: zero trust, zero cloud storage. Using browser Web Crypto, we hash her evidence on-device using SHA-256. Original photos never leave her browser. We build a tamper-evident hash chain stored in AES-GCM encrypted IndexedDB with a 200,000-iteration PBKDF2 key. If an attacker alters a logged record, our chain verification algorithm instantly pinpoints the exact broken link. Her chat screenshots are parsed in-browser via WebAssembly Tesseract OCR — offline and private.'

Step 3: Grounded Legal Guidance & Section 63 BSA (1:20 - 2:10)

'Instead of generative legal hallucination, Sakshya uses a strictly gated multilingual RAG engine across 18 provisions of the BNS, IT Act, and POC SO. Every legal provision links directly to India Code and carries a 'needs verification' notice. Finally, Sakshya generates three court-ready documents: a police complaint, a platform takedown request, and crucially, a preparation worksheet for the Section 63 BSA electronic evidence certificate, listing the exact SHA-256 hashes of her preserved files.'

Step 4: Victim Safety & Closing (2:10 - 3:00)

'If an abuser enters the room, pressing Escape twice triggers our Quick Exit, instantly clearing session memory and replacing the tab with Google. Sakshya is fully localized in English, Hindi, and Marathi, works offline as an installable PWA, and requires zero paid API keys. It transforms helplessness into a structured legal action pack. Thank you, and we welcome your questions.'

5. Comprehensive Judge Q&A; & Conceptual Deep-Dive

Anticipate these rigorous questions from technical, legal, and product judges. Here are the exact architectural answers:

Q1: How do you mathematically guarantee that images never leave the victim's device?

Answer: Through three distinct architectural barriers:

1. **Browser Memory Sandbox:** The file input event reads the file as an ArrayBuffer directly into the browser's JavaScript V8 heap. We invoke `crypto.subtle.digest('SHA-256', arrayBuffer)` on-device. The binary buffer is immediately garbage collected.
2. **Strict Network Payloads:** The API client (`Frontend/lib/api.ts`) only ever transmits JSON text payloads containing `{ fileName, fileHash, platform, note }`. Our Playwright automated tests sniff network packets and verify that neither binary data nor base64 image strings ever appear in HTTP requests.
3. **Backend Rejection:** The FastAPI `PrivacyBoundary` middleware inspects incoming HTTP headers. If `Content-Type` is not `application/json`, it rejects the call with HTTP 415. Pydantic schemas enforce `extra='forbid'`, rejecting any unexpected fields.

Q2: Why does the backend not have a database? Isn't state persistence necessary?

Answer: In victim privacy engineering, data you don't possess cannot be breached, subpoenaed, or leaked. A central database storing harassment reports or evidence hashes would become a catastrophic target for hackers and malicious actors. Instead, persistence is delegated entirely to the client's device using `Dexie IndexedDB`, encrypted with `AES-256-GCM`. The FastAPI backend is purely a stateless computation utility (NLP feature extraction, vector RAG retrieval, and ReportLab PDF compilation). Each HTTP request is self-contained, executed in memory, and immediately released.

Q3: Can an attacker modify the local timeline? What does the hash chain actually prove?

Answer: The hash chain proves internal integrity against tampering relative to the recorded state.

Each block entry digest is computed as `SHA256(index | timestamp | fileName | fileHash | platform | note | prevHash)`. Because each block embeds the cryptographic hash of the prior block, changing any byte in entry #1 alters its digest, invalidating entry #2's `prevHash`. Our verification algorithm iterates from index 1 to N, detecting the exact link where the chain breaks.

What it does NOT prove: We explicitly disclaim that a client-side chain cannot prove independent third-party time or absolute source authenticity against an attacker with full unlocked root device access who completely rewrites the chain. That is why our UI advises exporting the timeline PDF and sharing the digest with a trusted third party or advocate immediately.

Q4: What is Section 63 of Bharatiya Sakshya Adhiniyam, 2023 (BSA), and why is it important?

Answer: In July 2024, India replaced the Indian Evidence Act, 1872 with the Bharatiya Sakshya Adhiniyam, 2023 (BSA). Section 63 of the BSA replaces the famous Section 65B of the old act regarding the admissibility of electronic records in court. Section 63 mandates a formal certificate comprising two distinct parts: Part A (signed by the person in lawful control of the device/account) and Part B (signed by an authorized expert/forensic examiner).

Sakshya generates a Section 63 Preparation Worksheet that pre-populates all cryptographic file hashes, device details, and required statutory clauses. It helps victims and their advocates collect the exact parameters required by the court, preventing evidence from being rejected as inadmissible.

Q5: How does Quick Exit / Smart Exit work? Does it delete browser history?

Answer: Quick Exit is triggered either by clicking the top-right button or pressing `Escape` twice within 1.2 seconds. It executes three operations in sequence:

1. `lock()`: Terminates any running Tesseract WebAssembly worker, clears unencrypted React memory state (case details, OCR text, drafts).
2. Destroys the unlocked in-memory `CryptoKey` so the encrypted vault cannot be decrypted without re-entering the passphrase.
3. Calls `window.location.replace('https://www.google.com/')`. `location.replace` replaces the current entry in the session history, meaning clicking the browser's 'Back' button will not return to Sakshya.

Forensic Transparency: We explicitly disclose in the UI footer that Quick Exit does not delete browser cache, downloaded PDFs, or local `IndexedDB` files. True private browsing still requires using Incognito/Private mode.

Q6: How would Sakshya handle crowd control, viral surges, and DDoS attacks in production?

Answer: Sakshya's architecture is uniquely optimized for high-concurrency crowd control because the heavy computational work is decentralized:

1. Client-Side Offloading: Hashing (SHA-256), encryption (AES-GCM), database storage (IndexedDB), and image OCR (Tesseract WASM) run entirely on the client's CPU/GPU. The server does not process image bytes or store data.
2. Edge Caching of Static Shell: The Next.js frontend is fully exportable to Cloudflare CDN or Vercel Edge, absorbing unlimited traffic.
3. Stateless API Scalability: The FastAPI backend handles only tiny text JSON payloads (~1 KB each). It can be horizontally scaled in Docker containers behind Nginx/HAProxy with token-bucket rate limiting (e.g., 20 requests/minute per IP) to prevent denial-of-service abuse.
4. Deterministic Offline Fallback: If backend load spikes, users can still record evidence, run OCR, and verify vaults completely offline.

Q7: Why did you build a Progressive Web App (PWA) instead of a Native Android/iOS App?

Answer: In domestic abuse and cyberstalking scenarios, having an app icon called 'Sakshya Harassment Navigator' on a woman's phone is dangerous. If the abuser or family member inspects the phone, app store download histories and installed icons reveal that she is gathering evidence.

A PWA accessed via an Incognito browser tab leaves zero app store receipts. The user can bookmark it or install it discretely. Furthermore, web technologies ensure instant access across all device types (Android, iOS, Windows, macOS, Linux) without requiring app store approval.

Q8: How do you prevent LLM hallucinations from misinforming victims on the law?

Answer: By strictly constraining the AI's role in the architecture:

1. No Free-Form Legal Generation: The LLM is strictly prohibited from writing statutory text or inventing sections. All legal provisions are retrieved deterministically from our curated 18-provision corpus.
2. Constrained Explanation Choices: When generating the 'Why it applies' rationale, the LLM is given a discrete multiple-choice index of pre-audited explanations mapped to each corpus ID. It only selects the most relevant index.
3. Keyword Gate: If a query does not mention terms relevant to cyber harassment (e.g., queries about gardening or rent), the keyword filter intercepts the request and returns `no_match: true`, preventing false legal matches.
4. Zero-Key Mock Mode: In default mode, Sakshya uses deterministic rule-based NLP requiring no external API, ensuring 100% predictable output.

Q9: How does the system handle cases involving minors (under 18)?

Answer: Minor cases fall under the strict provisions of the POCSO Act, 2012 and Section 67B of the IT Act:

1. Mandatory Reporting Notice: Section 19 of POCSO mandates reporting child sexual exploitation to the Special Juvenile Police Unit or local police.
2. Childline 1098 Priority: Triage immediately surfaces 1098 Childline and urges involving a trusted guardian.
3. Take It Down vs StopNCII: StopNCII.org is strictly for adults (images taken at 18+). For minors, Sakshya directs the victim to NCMEC's Take It Down service (takeitdown.ncmec.org), which creates hashes for youth under 18.
4. Evidence Warning: The app warns: 'For a minor, seek specialist help on safe preservation. Do not download, forward, or redistribute sexual images.'

Q10: What is the roadmap for taking Sakshya from hackathon prototype to production?

Answer:

1. Direct StopNCII API Integration: Partnering with SWGfL / Meta to submit client-computed PDQ image hashes directly to StopNCII from the browser.
2. EXIF Metadata Cryptographic Binding: Extracting and hashing camera metadata (device serial, timestamp, GPS) alongside raw file bytes.
3. Advocate Network Handoff: End-to-end encrypted peer-to-peer package export (via WebRTC or QR code) directly to verified legal aid clinics.
4. India Code Live Synchronization: Automated scrapers to alert users of statutory amendments or commencement notifications.